

Algorithmes et Pensée Computationnelle

Structure de données, Itération et récursivité - Basiques

Le but de cette séance est de mettre en pratique les connaissances du cours sur les structures de données (tuples, listes, dictionnaires), l'itération et la récursivité. Au terme de cette séance, l'étudiant saura distinguer structures immuables et non immuables, et écrire des programmes utilisant itération et récursivité.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier "Exercices > Code". Le temps mentionné (🕒) est à titre indicatif.

1 Structures de données

Dans la première partie de cette section, les exercices devront être traités en Python. Dans la seconde partie, les exercices seront traités en Java.

1.1 Tuples

Pour rappel, les tuples sont des listes d'éléments immuables, ce qui signifie que ces listes ne peuvent pas être modifiées. L'une des particularités des tuples est la possibilité d'y inclure des données de types différents. Les tuples sont utiles pour stocker des données que l'on va réutiliser plus tard. En Python, pour créer un tuple, il suffit de définir une variable et de lui assigner des valeurs entre parenthèses et séparer les valeurs entre elles par des virgules :

> Exemple

```
1 mon_tuple = (1, "une chaîne de caractères", 3, 4)
```

Question 1: (🕒 5 minutes) Manipulation d'un tuple

Créez un tuple nommé `mon_tuple` contenant les chiffres 1,2,3,4 et 5. Affichez le 4ème élément de ce tuple.

💡 Conseil

Pour accéder à un élément d'un tuple, ou d'une liste, vous pouvez utiliser l'indexation. Comme pour accéder aux caractères des chaînes de caractères, utilisez `[]`. Les indices commencent à zéro.

> Solution

```
1 mon_tuple = (1, 2, 3, 4, 5)
2 print(mon_tuple[3])
```

Question 2: (🕒 5 minutes) Manipulation d'un tuple

Qu'affiche le code suivant ?

💡 Conseil

Vous pouvez contrôler si un élément (ou une sous-chaîne de caractères) est contenu dans un tuple, ou une liste, ou une chaîne de caractères en utilisant l'opérateur `in`. Si l'élément est dans la liste, la valeur booléenne `True` sera retournée. S'il n'y est pas, la valeur booléenne `False` sera retournée.

```

1 mon_tuple = (1,2,3,4,5,6)
2
3 if 6 in mon_tuple :
4     print("6 est contenu dans le tuple")
5 else :
6     print("6 n'est pas contenu dans le tuple")

```

💡 Conseil

Contrôlez si l'élément est contenu dans le tuple et regardez quelle branche sera prise.

>_ Solution

"6 est contenu dans le tuple"
6 est un élément de notre tuple, la condition sera donc remplie.

1.2 Listes

La différence entre une liste et un tuple (en Python) est que l'on peut modifier les éléments d'une liste. Par exemple, il est possible de remplacer un élément d'une liste par un autre tandis qu'il est impossible de le faire avec un tuple.

Pour créer une liste, il suffit de définir une variable avec des valeurs entre crochets :

>_ Exemple

```

1 ma_liste = [1,2,3,4,5]

```

Question 3: (🕒 5 minutes) Manipulation d'une liste - 1

Créez une liste nommée `ma_liste` contenant les nombres 1,2,3,4 et 5. Stockez le deuxième élément de la liste dans une variable nommée `element_2`, puis stockez la taille de la liste dans une variable nommée `taille_liste`. Affichez ces deux variables.

💡 Conseil

Comme pour les tuples, vous pouvez utiliser `[]` pour accéder à un élément, et la fonction `len()` pour obtenir le nombre d'éléments contenus dans la liste.

>_ Solution

```

1 ma_liste = [1,2,3,4,5]
2 element_2 = ma_liste[1]
3 taille_ma_liste = len(ma_liste)
4 print(element_2)
5 print(taille_ma_liste)

```

Question 4: (🕒 5 minutes) Manipulation d'une liste - 2

Qu'affiche le code suivant ?

```

1 ma_liste = [1,2,2,4,5]
2 ma_liste[2] = 3

```

```

3 ma_liste.append(6)
4 ma_liste.insert(0,0)
5 print(ma_liste)

```

Choisissez parmi les éléments suivants :

- 1, 2, 2, 4, 5, 6
- 0, 1, 2, 3, 4, 5
- 0, 1, 2, 3, 4, 5, 6
- 0, 1, 3, 2, 4, 5, 6

Conseil

En Python, vous pouvez accéder à chaque élément d'une liste en utilisant `[]` et modifier sa valeur. Pour ajouter un élément à la fin de la liste, utilisez la fonction `append()` et pour choisir l'endroit où vous désirez insérer votre nouvel élément, utilisez la fonction `insert()`.

>_ Solution

`[0, 1, 2, 3, 4, 5, 6]`
Le programme change le deuxième 2 pour un 3, ajoute un 6 à la fin de la liste, et un 0 au début de cette dernière.

1.3 Dictionnaires

Les dictionnaires sont des listes associatives, c'est-à-dire des listes qui lient une valeur à une autre. Dans un dictionnaire en papier, les mots sont liés à leur définition.

Dans un dictionnaire Python, les éléments sont liés par une relation dite `clé`, `valeur`. La clé étant le moyen de "retrouver" notre valeur dans notre dictionnaire. Par exemple, dans un dictionnaire en papier, nous pouvons retrouver une définition en cherchant le mot qui lui correspond, alternativement, nous pouvons retrouver le contenu d'une page d'un livre en utilisant le numéro de celle-ci, dans ce cas le numéro est la clé et le texte de la page, la valeur.

Pour créer un dictionnaire, il suffit de lister les couples `clé`, `valeurs` entre deux accolades :

```

1 elon_musk = {
2     "prénom": "Elon",
3     "nom": "Musk",
4     "age": 48,
5     "talents": ["programmation", "entrepreneuriat", "aéronautique"]
6 }
7

```

Question 5: (🕒 5 minutes) Manipulation d'un dictionnaire - 1

Créez un dictionnaire nommé `fr_eng` contenant les associations suivantes :

```

"chat": "cat", "chien": "dog", "oiseau": "bird", "poule": "chicken",
"papillon": "butterfly", "souris": "mouse", "ours": "bear",
"mouton": "sheep", "cochon": "pig"

```

Ce dictionnaire contient des mots en français (`clés`) associés à leur traduction en anglais (`valeurs`).

Accédez à la traduction du mot "souris", et stockez le résultat dans une variable nommée `souris_traduite`, puis calculez le nombre d'éléments contenus dans le dictionnaire et stockez le résultat dans une variable nommée `taille_fr_eng`. Affichez le contenu des deux variables que vous venez de créer.

💡 Conseil

Comme pour les listes, vous pouvez accéder aux éléments de dictionnaires en utilisant des crochets []. Seulement cette fois-ci, au lieu d'y mettre l'index, mettez-y la clé associée à l'élément auquel vous voulez accéder. Comme pour les listes, la fonction `len()` vous aidera à obtenir le nombre d'éléments dans le dictionnaire.

>_ Solution

```
1 fr_eng = {
2     "chat": "cat",
3     "chien": "dog",
4     "oiseau": "bird",
5     "poule": "chicken",
6     "papillon": "butterfly",
7     "souris": "mouse",
8     "ours": "bear",
9     "mouton": "sheep",
10    "cochon": "pig"
11 }
12 souris_traduite = fr_eng["souris"]
13 taille_fr_eng = len(fr_eng)
14 print(souris_traduite)
15 print(taille_fr_eng)
```

Question 6: (🕒 5 minutes) Manipulation d'un dictionnaire - 2

Qu'affiche le programme suivant ?

```
1 elon_musk = {
2     "prénom": "Elon",
3     "nom": "Musk",
4     "age": 48,
5     "talents": ["programmation", "entrepreneuriat",
6     "aéronautique"]
7 }
8 print(elon_musk["talents"][2])
9 print("aéronautique" in elon_musk["talents"])
```

Choisissez parmi les possibilités suivantes :

- programmation
True
- Musk
False
- 48
False
- aéronautique
True

💡 Conseil

L'élément associé à la valeur "talents" du dictionnaire `elon_musk` est une liste.

>_ Solution

```
aéronautique
True
```

En effet, via `elon_musk["talents"]`, on accède à la liste ["programmation", "entrepreneuriat", "aéronautique"]. Il est donc possible de manipuler cette liste comme une liste classique !

1.4 Structures de données (Java)

Dans cette partie, les exercices devront être effectués en Java.

Les syntaxes des principales structures de données abordées dans cette partie sont les suivantes :

Déclaration et affectation d'un tuple (array) en Java de type `Object[]` :

```
1 Object[] mon_tuple = {1, 2, 3, 4};
```

Déclaration et affectation d'un tuple (array) en Java de type `String[]` :

```
1 String[] mon_tuple = {"UNIL", "ACT"};
```

Pour la déclaration généralement, on écrit : **`T[] nom_variable`**, où **`T`** est le type d'éléments dans le tuple.

Déclaration et affectation d'une liste en Java :

```
1 List ma_liste = List.of(1, 2, 3, 4);
```

Déclaration et affectation d'un dictionnaire en Java :

```
1 HashMap mon_dictionnaire = new HashMap(Map.of("un", 1, "deux", 2));
```

>_ Informations utiles

— En Java, les listes sont immuables, comme les tuples en Python. Au cas où vous devriez modifier une liste, on vous recommande d'utiliser une liste liée (`LinkedList`). Une liste liée se déclare comme suit :

```
1 List liste = List.of(1, 2, 3, 4);
2 LinkedList ma_liste = new LinkedList(liste);
```

— En Java, les listes ne peuvent contenir que des éléments de même type.

Question 7: (🕒 10 minutes) Manipulation des listes en Java

Créez une liste nommée `ma_liste` contenant les nombres 1,2,3,4 et 5. Affichez le deuxième élément de la liste ainsi que la taille de la liste.

Créez une liste `ma_liste_m` liée à la liste `ma_liste`. Ajoutez le chiffre 6 à la fin de la liste, et le chiffre 0 au début de cette dernière.

Ajoutez ceci au début de votre code :

```
1 import java.util.List;
2 import java.util.LinkedList;
```

Ajoutez ceci à la fin de votre code :

```

1  for(int i=0;i<ma_liste_m.size();i++){
2      System.out.println(ma_liste_m.get(i));
3  }

```

💡 Conseil

Pour obtenir un élément d'une liste, il faut utiliser la fonction `.get(index)`. Pour obtenir la taille de la liste, utilisez la fonction `.size()`.

Pour ajouter un élément au début d'une `LinkedList`, vous devez utiliser `.addFirst(value)` et pour l'ajouter à la fin, vous devez utiliser `.addLast(value)`.

N'oubliez pas d'écrire votre programme à l'intérieur d'une fonction principale `main`.

```

1  public static void main(String[] args) {
2      // Ecrire votre code ici
3  }

```

>_ Solution

```

1  import java.util.List;
2  import java.util.LinkedList;
3
4  public class Question7 {
5
6      public static void main(String[] args) {
7          List ma_liste = List.of(1, 2, 3, 4);
8          System.out.println(ma_liste.get(1));
9          System.out.println(ma_liste.size());
10         LinkedList ma_liste_m = new LinkedList(ma_liste);
11         ma_liste_m.addFirst(0);
12         ma_liste_m.addLast(6);
13         for (int i = 0; i < ma_liste_m.size(); i++) {
14             System.out.println(ma_liste_m.get(i));
15         }
16     }
17 }

```

Question 8: (🕒 15 minutes) Manipulation de dictionnaires en Java

Créez un dictionnaire `mon_dictionnaire` contenant les associations suivantes :

```
("étudiants", 14000, ``enseignants``, 2300, ``collaborateurs``, 0)
```

Affichez le nombre d'étudiants. Obtenez la taille du dictionnaire et stockez le résultat dans une variable `taille_dictionnaire`. Affichez le contenu de `taille_dictionnaire`. Modifiez la valeur de `collaborateurs`, remplacez 0 par 950. Rajoutez un nouvel élément `pays` dans votre dictionnaire avec pour valeur 86.

Ajoutez ceci au début de votre code :

```

1      import java.util.HashMap;
2      import java.util.Map;

```

Ajoutez ceci à la fin de votre code :

```

1      for (String keys : mon_dictionnaire.keySet()) {

```

```

2         System.out.println(keys + " : " +
mon_dictionnaire.get(keys));
3     }

```

💡 Conseil

Comme pour les listes, accédez aux valeurs via la fonction `.get(key)` et à la taille via la fonction `.size()`.
Pour insérer / corriger un élément, utilisez la fonction `.put(key, value)`.

>_ Solution

```

1  import java.util.HashMap;
2  import java.util.Map;
3
4  public class Main {
5
6      public static void main(String[] args) {
7          HashMap<String, Integer> mon_dictionnaire = new
HashMap<String, Integer>(Map.of("étudiants", 14000,
"enseignants", 2300, "collaborateurs", 0));
8          System.out.println(mon_dictionnaire.get("étudiants"));
9          int taille_dictionnaire = mon_dictionnaire.size();
10         System.out.println(taille_dictionnaire);
11         mon_dictionnaire.put("collaborateurs", 950);
12         mon_dictionnaire.put("pays", 86);
13         for (String keys : mon_dictionnaire.keySet()) {
14             System.out.println(keys + " : " +
mon_dictionnaire.get(keys));
15         }
16     }
17 }

```

2 Itération

Quelques rappels de concepts théoriques :

- L'itération désigne l'action de répéter un processus (généralement à l'aide d'une boucle) jusqu'à ce qu'une condition particulière soit remplie.
- Il y a deux types de boucles qui sont majoritairement utilisées :

Boucle for : Une boucle `for` permet d'itérer sur un ensemble. Cet ensemble peut être une liste, un dictionnaire, une collection, ... La syntaxe d'une boucle en Python est la suivante `for nom_de_variable in` suivi du nom de l'élément sur lequel vous voulez itérer. La variable prendra la valeur de chaque élément dans la liste, un par un.

La syntaxe en Java est la suivante : `for (type nom_de_variable; condition de fin de boucle; incrémentation à chaque itération) suivi d'une accolade`. Il est également d'utiliser la syntaxe suivante pour itérer directement sur les éléments d'une liste : `for (var nom_de_variable : nom_de_la_liste)` ou `for (var nom_de_variable : nom_du_dictionnaire.keySet())` pour un dictionnaire

Boucle while : Les boucles `while` sont des boucles qui s'exécutent de façon continue jusqu'à ce qu'une condition soit remplie. La syntaxe est la suivante : En Python, on écrit d'abord `while`, suivi de la condition à atteindre.

En Java, il n'y a pas de différence majeure à part le fait qu'il faille mettre entre parenthèses la condition et les deux points sont remplacés par des accolades.

- La fonction `range(n)` permet de créer une liste de nombres de 0 à la valeur passée en argument moins 1 ($n-1$). Lorsqu'elle est combinée à une boucle `for`, on peut itérer sur une liste de nombres de 0 à $n-1$.
- Si vous avez fait une erreur dans votre code, il se peut que la boucle `while` ne remplisse jamais la condition lui permettant de sortir. On parle dans ce cas d'une boucle infinie. Ceci peut entraîner un crash de votre programme, voire même de votre ordinateur. Il faut toujours s'assurer qu'il y ait une condition valide dans une boucle `while`.

Question 9: (🕒 5 minutes) Boucle for en Python

Qu'affiche le code suivant ?

```
1         ma_liste = [1,2,3,4,5]
2         for c in ma_liste :
3             print(c)
4
```

Choisissez parmi les possibilités suivantes :

- 1 2 3 4 5
- 5 4 3 2 1
- 3 2 1 5 4
- 4 3 5 1 2

💡 Conseil

Faites les étapes de la boucle une après l'autre et voyez ce que vous obtenez (avec chaque valeur).

>_ Solution

1 2 3 4 5

Question 10: (🕒 5 minutes) Boucle for en Java

Qu'affiche le code suivant ?

```
1         List ma_liste = List.of(2,3,4,5,6);
2         for(int i=0;i<ma_liste.size();i++) {
3             System.out.print(ma_liste.get(i));
4         }
5
```

Choisissez parmi les possibilités suivantes :

- 5 4 3 2 1
- 1 2 3 4 5
- 4 3 5 1 2
- 2 3 4 5 6

 Conseil

Faites les étapes de la boucle une après l'autre et voyez ce que vous obtenez (avec chaque valeur).

 Solution

2 3 4 5 6

Question 11: (🕒 5 minutes) **Boucle while en Python**

Qu'affiche le code suivant ?

```
1         i = 0
2         while i<5:
3             print(i)
4             i += 1
5
```

Choisissez parmi les possibilités suivantes :

- 0 1 2 3 4
- 1 2 3 4 5
- 4 3 2 1 5
- 2 3 4 0 1

 Conseil

Faites les étapes de la boucle une après l'autre et voyez ce que vous obtenez (avec chaque valeur).

 Solution

0 1 2 3 4

Question 12: (🕒 5 minutes) **Boucle while en Java**

Qu'affiche le code suivant ?

```
1         int i = 0;
2         while(i<5) {
3             i++;
4             System.out.print(i + " ");
5         }
6
```

Choisissez parmi les possibilités suivantes :

- 0 1 2 3 4
- 1 2 3 4 5
- 4 3 2 1 0
- 5 4 3 2 1

 Conseil

Faites les étapes de la boucle une après l'autre et voyez ce que vous obtenez (avec chaque valeur).

 Solution

1 2 3 4 5

Question 13: (🕒 5 minutes) **Boucle for et fonction range() - 1**

Qu'affiche le code suivant ?

```

1         ma_liste = ["J'aime", "Python"]
2         for x in range(len(ma_liste)):
3             print(x)
4             print(ma_liste[x])
5
6         #0
7         #J'aime
8         #1
9         #Python

```

Choisissez parmi les possibilités suivantes :

- 0 J'aime 1 Python
- J'aime 1 Python 2
- J'aime 0 Python 1
- 1 J'aime 2 Python

Conseil

Faites les étapes de la boucle une après l'autre et voyez ce que vous obtenez (avec chaque valeur).

>_ Solution

```

0
J'aime
1
Python

```

Dans les exercices suivants, les programmes doivent être écrits en Python.

Question 14: (🕒 5 minutes) Boucle for et fonction range() - 2

En utilisant la fonction `range()` et une boucle `for`, calculez la somme des entiers de 0 à 20 et affichez le résultat.

Conseil

- Lorsque `range(n)` est combinée à une boucle `for`, on peut itérer sur une liste de nombres de 0 à n-1.
- Déclarer une variable en dehors de la boucle. Au terme de l'itération, cette variable contiendra la somme des entiers de 0 à 20.

>_ Solution

```

1 somme = 0
2 for i in range(21):
3     somme += i    # A chaque iteration on rajoute i à somme.
4 print(somme)

```

Question 15: (🕒 10 minutes) Boucle while et input()

À l'aide d'une boucle `while`, demandez à l'utilisateur de rentrer une valeur. Tant que cette valeur ne correspond pas à 10, le programme redemande à l'utilisateur une nouvelle valeur.

💡 Conseil

- Définir à l'extérieur de la boucle une variable de type booléen utilisée pour le test dans `while`.
- Utilisez la fonction `input()` pour récupérer la saisie de l'utilisateur.
- La fonction `input()` retourne un `str` (chaîne de caractères), pensez à changer son type.

>_ Solution

```
1 bool_test = True
2 while bool_test:                                #
    Tant que bool_test est True, la boucle continue
3     test_value = int(input("Veuillez entrer un entier")) #
    Pour que le test effectué à la ligne suivante ne soit pas
    toujours juste
4
    # (comparaison entre un string et un entier), il faut
    changer le type de l'input.
5     bool_test = not(test_value == 10)           #
    On veut sortir de la boucle si la test_value est 10. Pour
    sortir il faut que bool_test soit
6     if bool_test == True:                       #
    False. D'où l'utilisation de not qui transforme true en
    false et vice versa.
7         print("Ce n'est pas le bon entier.")
8 print("Bravo !")
```

Question 16: (🕒 5 minutes) Listes en compréhension (Python)

Définissez une liste `nombre` contenant l'ensemble des nombres compris entre 0 et 10. Ensuite, créez une liste en compréhension qui parcourt `nombre`, récupère tous les nombres pairs et les stocke dans une nouvelle variable de type `list` nommée `nombre_pairs`. Affichez le contenu de la variable `nombre_pairs`.

💡 Conseil

Pour générer une liste de nombres compris entre 0 et 10, vous pouvez utiliser l'instruction suivante : `nombre = list(range(10))`.

>_ Solution

```
1 nombre = list(range(10))
2 nombre_pair = [x for x in nombre if x%2==0]
3 print(nombre_pair)
```

Question 17: (🕒 10 minutes) Itérateurs (Java)

En Java, définissez une liste `nombre` contenant l'ensemble des nombres compris entre 0 et 10. En utilisant un `Iterator`, supprimez tous les nombres impairs présents dans `nombre`. Affichez le contenu de `nombre`.

💡 Conseil

La diapositive 25 du cours montre comment supprimer un élément d'une liste en utilisant un `iterator`.

>_ Solution

```
1 import java.util.ArrayList;
2 import java.util.Arrays;
3 import java.util.Iterator;
4 import java.util.List;
5
6 class Scratch {
7     public static void main(String[] args) {
8         List<Integer> nombres = new
9             ArrayList<>(Arrays.asList(0, 1, 2, 3, 4, 5, 6, 7, 8, 9));
10         Iterator<Integer> iter = nombres.iterator();
11
12         while(iter.hasNext()) {
13             if (iter.next()%2==1) {
14                 iter.remove();
15             }
16         }
17         System.out.println(nombres);
18     }
19 }
```

3 Récursivité

La récursivité est le fait d'appeler une fonction de façon récursive c'est-à-dire une fonction qui s'appelle elle-même de façon directe ou indirecte.

Question 18: (🕒 5 minutes) Lecture de code - 1

Qu'affiche le programme suivant ?

```
1 def fonction_p(x) :
2     print(x)
3     if x == 0 :
4         pass
5     else :
6         fonction_p(x-1)
7
8 fonction_p(5)
```

Choisissez parmi les possibilités suivantes :

- 4 3 2 1 0
- 5 4 3 2 1 0
- 0 1 2 3 4
- 1 2 3 4 5 0

>_ Solution

5 4 3 2 1 0

Question 19: (🕒 5 minutes) Lecture de code - 2

Qu'affiche le programme suivant ?

```

1 def fonction_p(x) :
2     if x == 0 :
3         pass
4     else :
5         fonction_p(x-1)
6         print(x)
7
8 fonction_p(5)

```

Choisissez parmi les possibilités suivantes :

- 4 3 2 1 0
- 5 4 3 2 1
- 0 1 2 3 4
- 1 2 3 4 5

Conseil

- La principale différence entre ces 2 programmes est l'endroit où est placé le `print()`.
- Reproduire étape par étape le développement sur un papier pourrait aider.

Solution

1 2 3 4 5

Question 20: 10 minutes) **Fonction factoriel()**

Ecrivez la fonction `factoriel()` suivant une approche récursive.

Pour rappel, cette fonction prend un entier et retourne le factoriel de ce dernier tel que :

$\text{factoriel}(1) = 1$, $\text{factoriel}(2) = 1 * 2 = 2$, $\text{factoriel}(3) = 1 * 2 * 3 = 6$, $\text{factoriel}(4) = 1 * 2 * 3 * 4 = 24, \dots$

Conseil

On peut écrire cette fonction comme suit :

$$f(n) = f(n) * f(n - 1) \text{ si } n > 0$$

$$f(n) = 1 \text{ si } n = 0$$

>_ Solution

Python :

```
1 def factoriel(x) :  
2     if x == 0:  
3         return 1  
4     else :  
5         return x * factoriel(x-1)
```

Java :

```
1 public static int factoriel(int x){  
2     if (x == 0){  
3         return 1;  
4     }  
5     else{  
6         return x*factoriel(x-1);  
7     }  
8 }  
9 public static void main(String[] args) {  
10     System.out.println(factoriel(5));  
11 }
```